

Lecture 3: Data Types & Variables in C#

- [Lecture 3: Data Types & Variables in C](#)
 - [1. What is a Data Type?](#)
 - [2. How Many Data Types Are There in C#?](#)
 - [A. Integer Types \(Whole Numbers\) — 8 types](#)
 - [B. Floating-Point Types \(Decimal Numbers\) — 3 types](#)
 - [C. Other Simple Types — 4 types](#)
 - [3. Finding the Size of Data Types in C](#)
 - [Code: Printing the Size of Every Data Type](#)
 - [Bonus Code: Checking the Min/Max Range of a Type](#)
 - [4. What is a Variable? \(With Real-World Example\)](#)
 - [Real-World Example: A Bank Account](#)
 - [Another Real-World Example: A Traffic Light](#)
 - [Anatomy of a Variable Declaration](#)
 - [Variable Naming Rules \(Important!\)](#)
 - [5. Value Types vs. Reference Types — Why It Matters for Variables](#)
 - [6. Handy Practice Codes \(Strengthen Your Concepts\)](#)
 - [Practice 1: Declare and Print All Basic Types](#)
 - [Practice 2: Swap Two Variables \(Classic Interview Question\)](#)
 - [Practice 3: Type Conversion in Action](#)
 - [Practice 4: Simple Calculator Using Variables](#)
 - [Practice 5: var — Let C# Infer the Type](#)
 - [Practice 6 \(Challenge\): Fahrenheit to Celsius Converter](#)
 - [7. Quick Recap \(Key Takeaways\)](#)
 - [8. What's Next?](#)

Lecture 3: Data Types & Variables in C

Continuing from Lecture 2 — now that you can write and run a C# program, it's time to understand what actually goes *inside* it: data types and variables, the raw material of every program you'll ever write.

1. What is a Data Type?

A **data type** tells the compiler (and the computer) **what kind of value** a piece of data is, **how much memory** to reserve for it, and **what operations** are valid on it.

Simple definition: > A data type is a classification that specifies what kind of value a variable can hold — a number, a piece of text, a true/false value, etc. — and how much memory that value needs.

Analogy: Think of data types like different sizes of containers in a kitchen. You wouldn't store a single

grain of rice in a giant drum, and you wouldn't try to stuff a sack of flour into a teaspoon. Similarly, a `bool` (true/false) needs a tiny "container" (1 byte), while a `double` (decimal number) needs a much bigger one (8 bytes). Choosing the right data type means using memory efficiently and avoiding data loss.

Why data types matter:

1. **Memory efficiency** – The compiler reserves exactly the right amount of memory, no more, no less.
2. **Type safety** – C# is a **strongly-typed** language (mentioned back in Lecture 1), meaning you can't accidentally store text in a number variable — the compiler catches this at compile time, before your program even runs.
3. **Correct operations** – The compiler knows `5 + 3` should do math, while `"5" + "3"` should join text, purely based on data type.
4. **Predictable behavior** – Every developer reading your code instantly knows what kind of value to expect.

2. How Many Data Types Are There in C#?

C# data types fall into **two major categories**:

Category	Stored In	Examples	Key Trait
Value Types	Stack (directly holds the value)	<code>int</code> , <code>double</code> , <code>bool</code> , <code>char</code> , <code>struct</code>	Fast, copied by value
Reference Types	Heap (variable holds a reference/address)	<code>string</code> , <code>class</code> , <code>array</code> , <code>object</code> , <code>interface</code>	Flexible, copied by reference

Within **Value Types**, C# provides **15 built-in (predefined) simple types**, commonly grouped as:

A. Integer Types (Whole Numbers) — 8 types

Type	Size	Range	Example
<code>sbyte</code>	1 byte	-128 to 127	Signed tiny numbers
<code>byte</code>	1 byte	0 to 255	Unsigned tiny numbers
<code>short</code>	2 bytes	-32,768 to 32,767	Small counters
<code>ushort</code>	2 bytes	0 to 65,535	Small unsigned counters
<code>int</code>	4 bytes	~-2.1 billion to 2.1 billion	Most common integer type
<code>uint</code>	4 bytes	0 to ~4.2 billion	Unsigned whole numbers
<code>long</code>	8 bytes	~-9.2 quintillion to 9.2 quintillion	Very large numbers (e.g., population counts)
<code>ulong</code>	8 bytes	0 to ~18.4 quintillion	Very large unsigned numbers

B. Floating-Point Types (Decimal Numbers) — 3 types

Type	Size	Precision	Example Use
float	4 bytes	~6-9 digits	Simple decimal math, game coordinates
double	8 bytes	~15-17 digits	Default choice for decimals (most common)
decimal	16 bytes	~28-29 digits	Money/financial calculations (very high precision)

C. Other Simple Types — 4 types

Type	Size	Holds	Example
char	2 bytes	A single Unicode character	'A', '9', '\$'
bool	1 byte	true or false only	Login status, feature flags
string*	Varies (reference type)	Sequence of characters (text)	"Hello World"
object*	Varies (reference type)	Can hold any data type (base of all types)	Generic storage

(`string` and `object` are technically **reference types**, not value types — but they're included here because they're used just as commonly as the value types above.)

So, in total: C# has **15 predefined value types + string + object = 17 commonly taught built-in types**, though `string` and `object` behave differently under the hood (reference types, as covered in the table in Section 2). Beyond these built-ins, C# also lets you create your own types using `struct`, `class`, `enum`, and `interface` — but those are custom types, not predefined ones, and we'll study them in later OOP-focused lectures.

3. Finding the Size of Data Types in C

C# gives you a built-in operator, `sizeof()`, and a set of static properties on each type (`.MaxValue`, `.MinValue`) to inspect these details directly instead of memorizing tables.

Code: Printing the Size of Every Data Type

```
Console.WriteLine("---- Size of C# Data Types (in bytes) ----");
Console.WriteLine($"sbyte : {sizeof(sbyte)} byte");
Console.WriteLine($"byte : {sizeof(byte)} byte");
Console.WriteLine($"short : {sizeof(short)} bytes");
Console.WriteLine($"ushort : {sizeof(ushort)} bytes");
Console.WriteLine($"int : {sizeof(int)} bytes");
```

```

Console.WriteLine($"uint    : {sizeof(uint)} bytes");
Console.WriteLine($"long    : {sizeof(long)} bytes");
Console.WriteLine($"ulong   : {sizeof(ulong)} bytes");
Console.WriteLine($"float   : {sizeof(float)} bytes");
Console.WriteLine($"double  : {sizeof(double)} bytes");
Console.WriteLine($"decimal: {sizeof(decimal)} bytes");
Console.WriteLine($"char    : {sizeof(char)} bytes");
Console.WriteLine($"bool    : {sizeof(bool)} byte");

```

Sample Output:

```

---- Size of C# Data Types (in bytes) ----
sbyte  : 1 byte
byte   : 1 byte
short  : 2 bytes
ushort : 2 bytes
int     : 4 bytes
uint    : 4 bytes
long    : 8 bytes
ulong   : 8 bytes
float   : 4 bytes
double  : 8 bytes
decimal: 16 bytes
char    : 2 bytes
bool    : 1 byte

```

Important note: `sizeof()` only works directly on **value types**. It will not work on `string` because `string` is a reference type with a variable, dynamic length — its size depends on how many characters it holds, not a fixed number of bytes.

Bonus Code: Checking the Min/Max Range of a Type

```

Console.WriteLine($"int    range: {int.MinValue} to {int.MaxValue}");
Console.WriteLine($"long   range: {long.MinValue} to {long.MaxValue}");
Console.WriteLine($"double range: {double.MinValue} to {double.MaxValue}");
Console.WriteLine($"char   range: {(int)char.MinValue} to {(int)char.MaxValue}");

```

This is extremely useful in real projects — instead of memorizing “int goes up to ~2.1 billion,” you just ask C# directly.

4. What is a Variable? (With Real-World Example)

A **variable** is a **named storage location in memory** that holds a value, and whose value **can change** while the program runs.

Simple definition: > A variable is a labeled box in memory where you store a piece of data so you can use it, change it, or refer to it later by name.

Real-World Example: A Bank Account

Imagine your **bank account balance**. It’s not a fixed number written once — it’s a value that changes constantly:

- You deposit ₹5,000 → balance becomes 5,000
- You withdraw ₹1,200 → balance becomes 3,800
- Interest is added → balance becomes 3,850

The **bank account balance** is exactly like a variable: it has a **name** (`balance`), a **type** (`decimal` , since it deals with money), and a **value that changes over time**, even though the “container” itself (the account) stays the same.

```
decimal balance = 5000;
Console.WriteLine($"Initial balance: {balance}");

balance = balance - 1200; // withdrawal
Console.WriteLine($"After withdrawal: {balance}");

balance = balance + 50; // interest added
Console.WriteLine($"After interest: {balance}");
```

Output:

```
Initial balance: 5000
After withdrawal: 3800
After interest: 3850
```

Another Real-World Example: A Traffic Light

A traffic light’s current color is a **variable** too:

```
string trafficLight = "Red";
Console.WriteLine($"Cars must: {(trafficLight == "Red" ? "Stop" : "Go")}");

trafficLight = "Green";
Console.WriteLine($"Cars must: {(trafficLight == "Red" ? "Stop" : "Go")}");
```

The **name** of the variable (`trafficLight`) stays constant in your code, but the **value it holds** (“Red”, then “Green”) changes — that’s the entire point of a variable.

Anatomy of a Variable Declaration

```
int    age    = 21;
//  ↑      ↑      ↑
// type  name  value
```

Part	Meaning
<code>int</code>	The data type — tells C# this variable will only ever hold whole numbers.
<code>age</code>	The identifier/name — how you refer to this value later in your code.
<code>= 21</code>	The initialization — assigning an initial value using <code>=</code> (the assignment operator).

Variable Naming Rules (Important!)

1. Must start with a letter or underscore (`_`) — never a digit.

2. Can contain letters, digits, and underscores — no spaces or special symbols.
3. C# is **case-sensitive** — `age`, `Age`, and `AGE` are three different variables.
4. Cannot use **reserved keywords** (`int`, `class`, `string`, etc.) as variable names.
5. Convention: use **camelCase** for variable names (`firstName`, `totalPrice`), and make names **descriptive** (`age` is better than `a`).

5. Value Types vs. Reference Types — Why It Matters for Variables

This is one of the most important beginner concepts, and it directly affects how variables behave when you copy them.

```
// VALUE TYPE example
int x = 10;
int y = x;    // y gets a COPY of x's value
y = 20;
Console.WriteLine($"x = {x}, y = {y}");    // x = 10, y = 20 (independent)

// REFERENCE TYPE example
int[] arr1 = { 1, 2, 3 };
int[] arr2 = arr1;    // arr2 points to the SAME array in memory
arr2[0] = 99;
Console.WriteLine($"arr1[0] = {arr1[0]}");    // arr1[0] = 99 (changed too!)
```

Why this happens: `int` is a value type, so `y = x` copies the actual number. But `int[]` is a reference type, so `arr2 = arr1` just copies the **address** — both variables now point to the same data in memory. This exact distinction is why the table in Section 2 separates “Stack” (value types) from “Heap” (reference types) — it’s not just theory, it changes real program behavior.

6. Handy Practice Codes (Strengthen Your Concepts)

Practice 1: Declare and Print All Basic Types

```
int wholeNumber = 42;
double price = 199.99;
char grade = 'A';
bool isPassed = true;
string studentName = "Riya";

Console.WriteLine($"Name: {studentName}, Grade: {grade}, Price: {price}, Passed: {isPassed}, Roll No: {wholeNumber}");
```

Practice 2: Swap Two Variables (Classic Interview Question)

```
int a = 5, b = 10;
Console.WriteLine($"Before swap: a = {a}, b = {b}");

int temp = a;
```

```

    a = b;
    b = temp;

    Console.WriteLine("After swap: a = {a}, b = {b}");

```

Practice 3: Type Conversion in Action

```

    int wholeNum = 9;
    double converted = wholeNum;    // implicit: int -> double (safe, no data loss)
    Console.WriteLine($"Implicit: {converted}");

    double decimalNum = 9.75;
    int truncated = (int)decimalNum;    // explicit cast: double -> int (loses .75!)
    Console.WriteLine($"Explicit cast: {truncated}");

```

Practice 4: Simple Calculator Using Variables

```

    Console.Write("Enter first number: ");
    double num1 = Convert.ToDouble(Console.ReadLine());

    Console.Write("Enter second number: ");
    double num2 = Convert.ToDouble(Console.ReadLine());

    double sum = num1 + num2;
    double product = num1 * num2;

    Console.WriteLine($"Sum: {sum}");
    Console.WriteLine($"Product: {product}");

```

Practice 5: var — Let C# Infer the Type

```

    var city = "Mumbai";    // compiler infers: string
    var population = 20000000;    // compiler infers: int
    var isCapital = false;    // compiler infers: bool

    Console.WriteLine($"{city}, Population: {population}, Capital: {isCapital}");

```

Note: `var` doesn't make C# "loosely typed" — the compiler still locks in the actual type (`string`, `int`, `bool`) at compile time based on the value you assign. You just save yourself from typing the type name explicitly. This is different from languages like Python, where a variable's type can change freely at runtime.

Practice 6 (Challenge): Fahrenheit to Celsius Converter

```

    Console.Write("Enter temperature in Fahrenheit: ");
    double fahrenheit = Convert.ToDouble(Console.ReadLine());

    double celsius = (fahrenheit - 32) * 5 / 9;

    Console.WriteLine($"{fahrenheit}°F is equal to {celsius:F2}°C");

```

(The `:F2` inside the interpolation formats the number to 2 decimal places — a small preview of formatting, which we'll cover properly later.)

7. Quick Recap (Key Takeaways)

- A **data type** defines what kind of value a variable can hold and how much memory it needs.
 - C# has **8 integer types, 3 floating-point types, and 4 other simple types** (`char` , `bool` , `string` , `object`) — split across **value types** (stack) and **reference types** (heap).
 - Use `sizeof(type)` to check memory size, and `.MinValue` / `.MaxValue` to check the range of any numeric type directly through code.
 - A **variable** is a named, changeable storage location — like a bank balance or a traffic light color, its name stays the same while its value changes.
 - **Value types are copied, reference types share memory** — a small distinction with big consequences for how your program behaves.
 - `var` lets the compiler infer the type for you, but C# remains strongly-typed underneath.
-

8. What's Next?

In Lecture 4, we'll cover: 1. Operators in C# (arithmetic, relational, logical, assignment) 2. Operator precedence and expressions 3. Type casting in more depth (implicit vs. explicit vs. `Convert` class vs. `Parse` / `TryParse`) 4. Constants using `const` and `readonly`

Tip for students: Open VS Code right now and type out every practice code above yourself — especially Practice 3 (type conversion) and Practice 5 (`var`). These two trip up almost every beginner in interviews, so build the muscle memory early.